Praktikum z programování 1
Protokol č. 2

U

Téma: Výběr tématu, cílové skupiny a popis plánovaného řešení a zdrojů

Vyučující: doc. RNDr. Petr Šaloun, Ph.D., Katedra technické a informační výchovy, PdF UPOL

Jméno a příjmení studenta: Vondráková Aneta

Datum vypracování: 4.11.2025

Témata k prostudování:

Moodle k předmětu PP1, výukové materiály k Pythonu, zejména knihovny

Zadání:

Na základě popisu zadání schváleného vyučujícím a sepsaného v Protokolu 1 popište řešení a zdokumentujte program, testovací vstupní a výstupní data a diskutujte použité algoritmy a postupy tak, aby byl postup jasný a doložili sjte funkčnost programu i použité algoritmy a postupy a tyto případně porovali s jiným dostupným řešením.
Součástí řešení může být například .txt soubor s linky na zdroje, které vás inspirovaly, popis github prostředí, odkud si stáhnete zdrojové kódy, licence… na svůj počítač a použijete je pro řešení… spolu se zdrojovým textem(-y) programu, testovacími daty s tímto protokolem 2 pojmenujte podle pokynů popsaných níže) zabalte do jediného ZIP archivu pojmenovaného *ID-2.zip*, jde ID je Vaše školní ID, např. *novakp-2.zip*.
Odevzdání prvního protokolu ID-2.ZIP bude pravděpodobně obsahovat soubory:

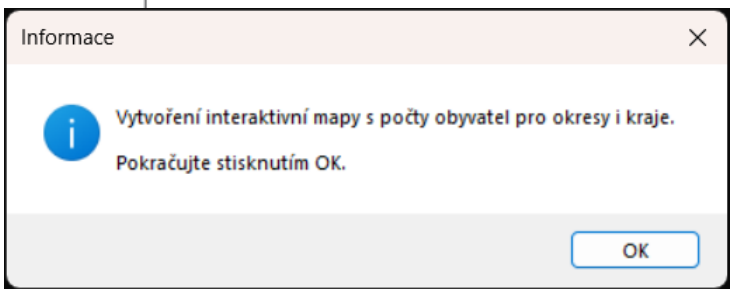
- ID-protokol-PP1-2.odt (případně .docx) je tento soubor pojmenovaný podle vzoru, a vyplněný podle požadavků, doplněný, screenshoty obrazovek a vaším komentářem, linky na zdroje, dokumentaci, popisy a návody, které jste jste našli a používáte jako východisko/inspiraci, použili anebo použít plánujete.

Výsledek konzultujte v případě potřeby ve cvičení s vyučujícím. Výsledek odevzdávejte výhradně do moodle pro daný předmět a protokol 2. Jiný způsob odevzdání nebude akceptován.

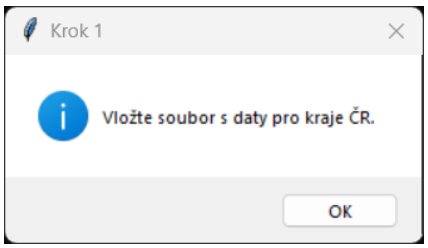
Návrh řešení:

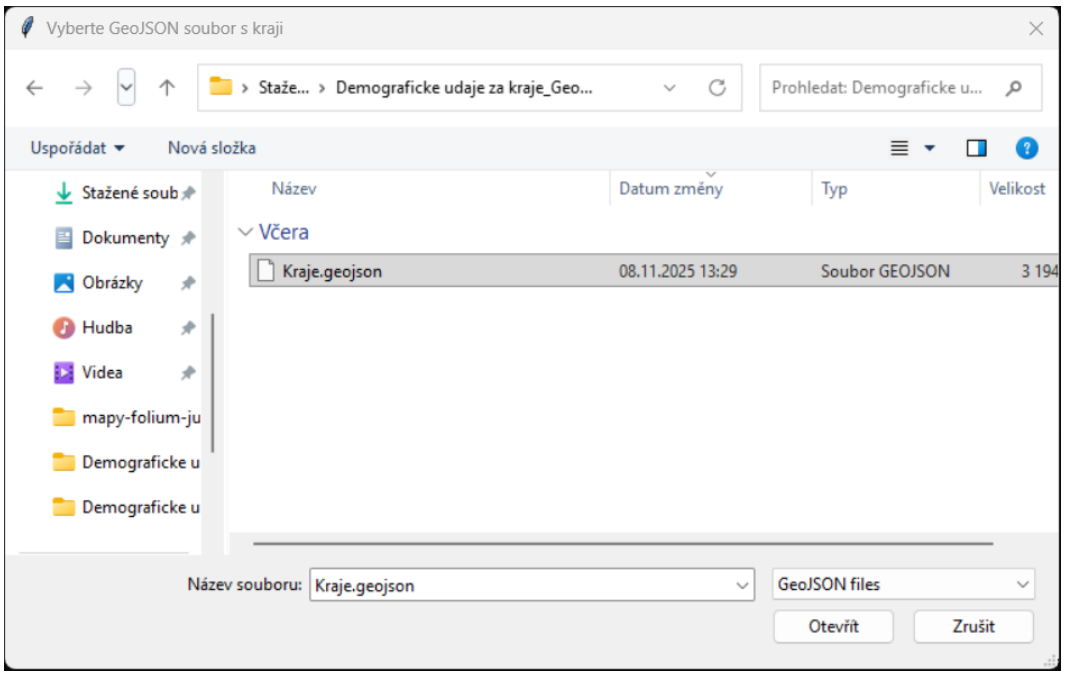
Protokol byl použit i do předmětu DPK. Odprezentováno 25.11.2025.

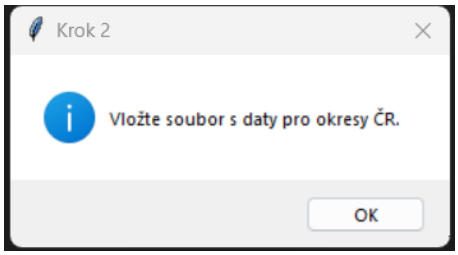
Cílem programu je načíst a zobrazit přehled krajů a okresů dle počtu obyvatel. Uživatel má možnost vybrat data ve formátu GeoJSON, která obsahují údaje o počtu obyvatel, žen a mužů. U souboru s kraji je i rozloha kraje. Nutností je, aby soubor obsahoval data se souřadnicemi – hranice krajů i okresů. Uživateli se zobrazí i body s krajskými městy.
Po spuštění programu vyskočí dialogové okno s informacemi o programu.



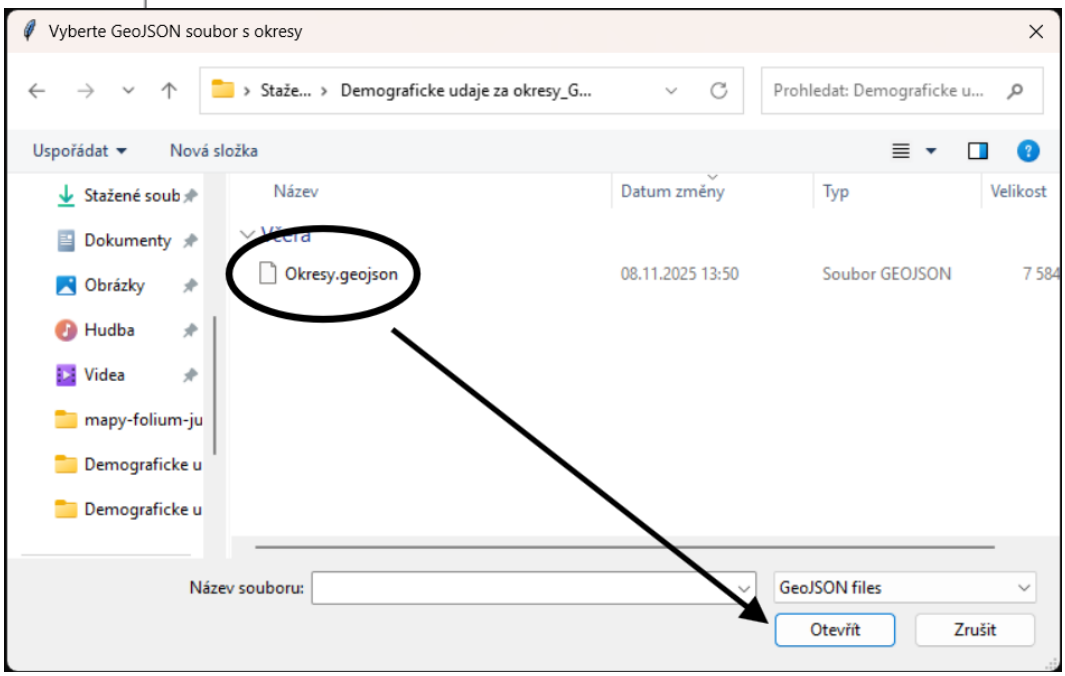
Vyskočí další dialogová okna pro vložení potřebných souborů – typ GeoJSON, JSON.



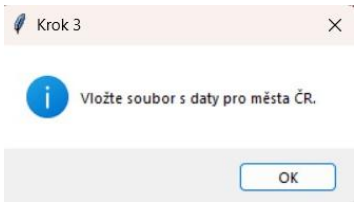




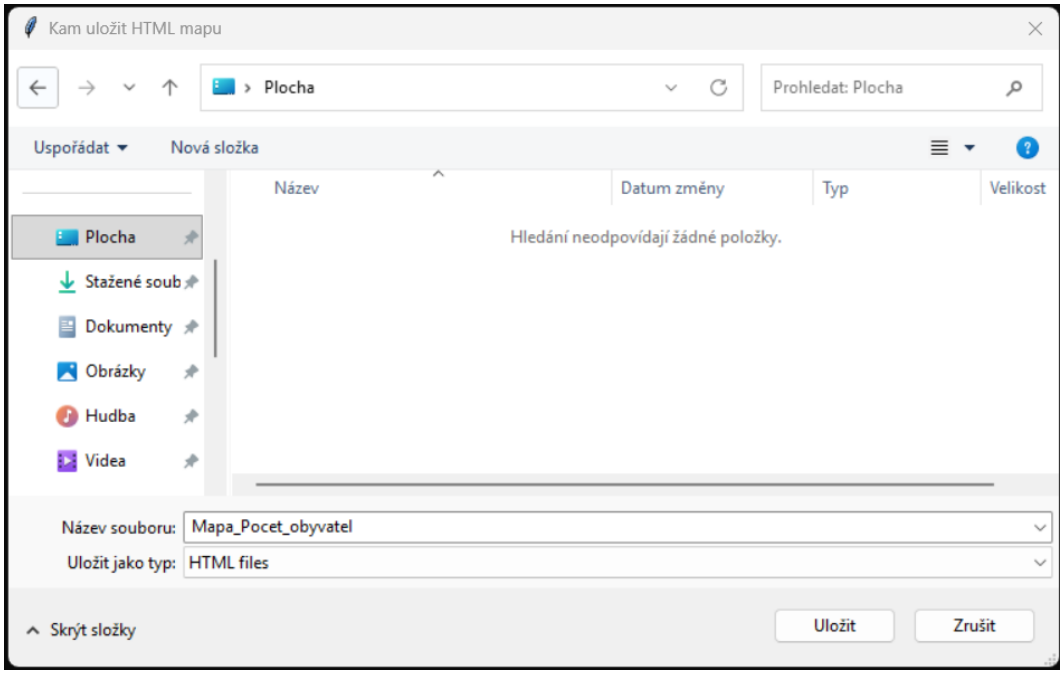
Kontakt: petr.saloun@upol.cz



Posledním krokem je vložení dat o krajských městech pro správné zobrazení bodů.

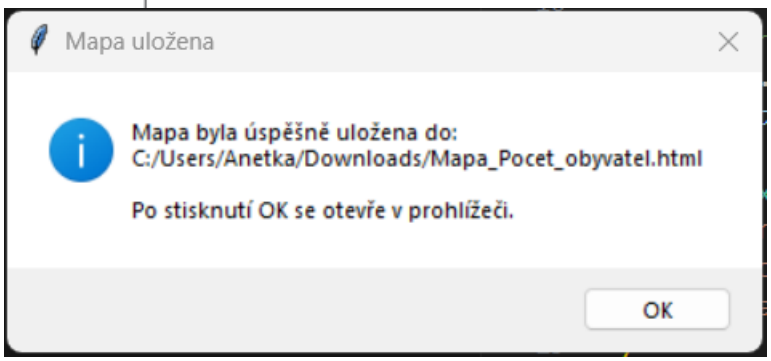


Dál vybereme místo uložení. Předvyplněný název vytvořeného souboru je Mapa_Pocet_obyvatel a ukládá se jako HTML.

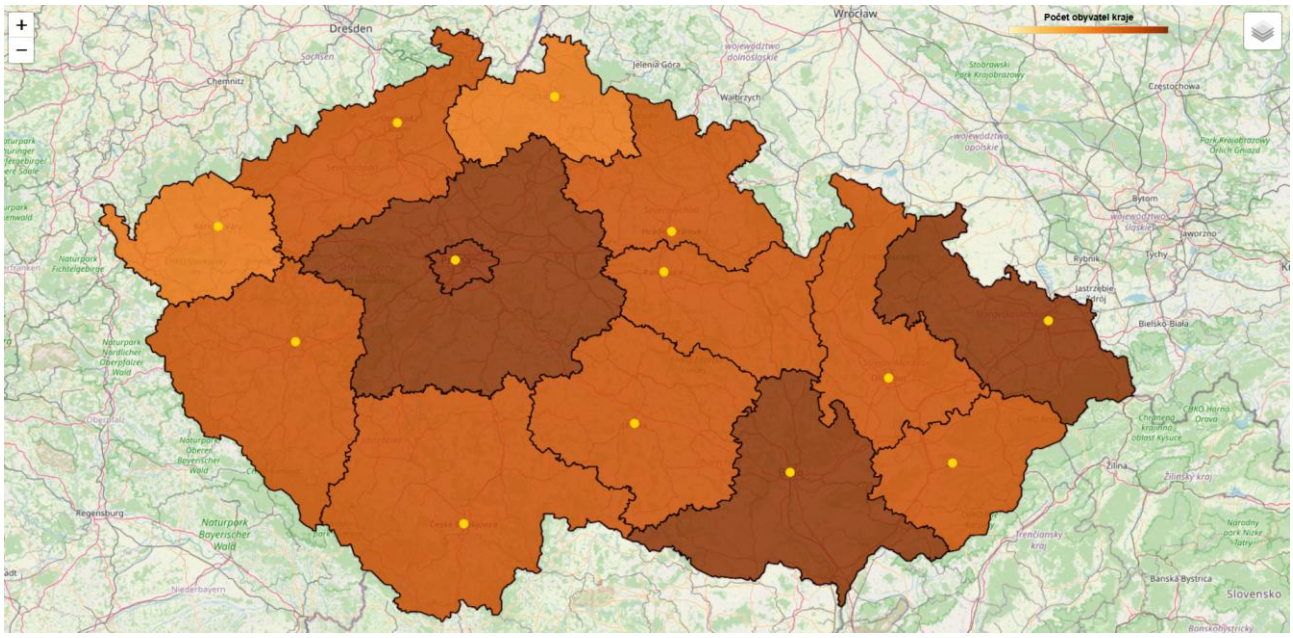


Dál se zobrazí informační okno s místem uložení a instrukcemi.

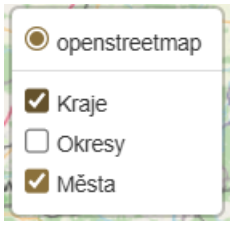
Kontakt: petr.saloun@upol.cz



Defaultně je nastaveno zobrazení pro kraje. Jednotlivé kraje jsou zbarveny dle počtu obyvatel – legenda v pravém horním rohu.

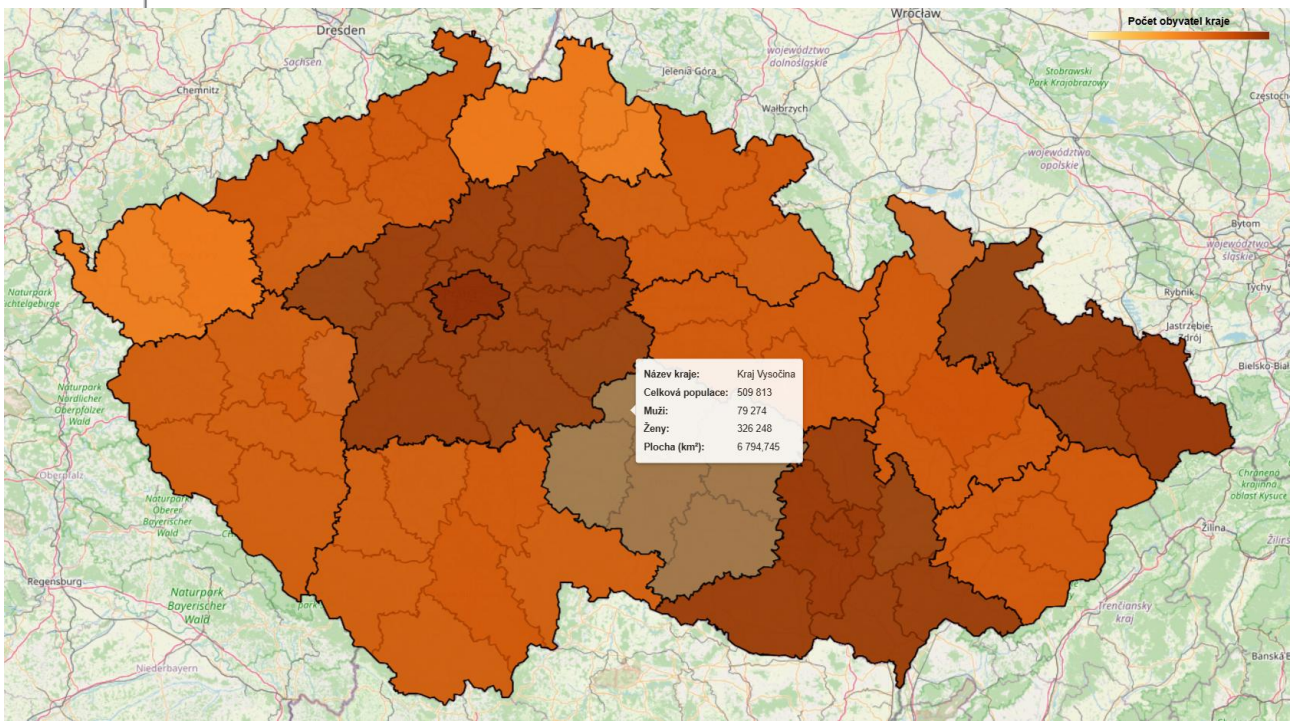


Můžeme přepínat mezi členěním ČR na kraje nebo okresy a zapínat zobrazení krajských měst.

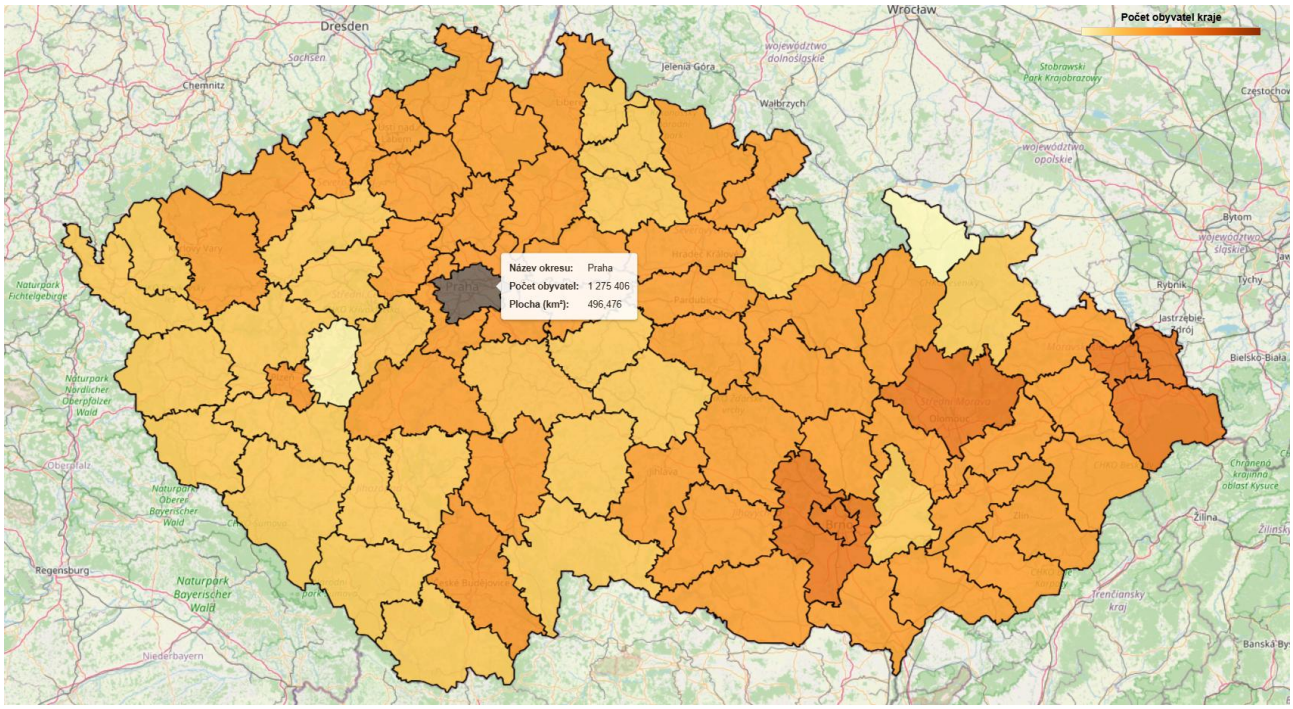


Po najetí myší na kraj se barevně odliší od ostatních a vyjede nám informační okno s názvem kraje, celkovým počtem obyvatel, počet žen a mužů v kraji a plocha kraje.

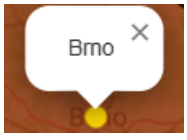
Kontakt: petr.saloun@upol.cz



Pro okresy se zobrazí název okresu, počet obyvatel a rozloha daného okresu.

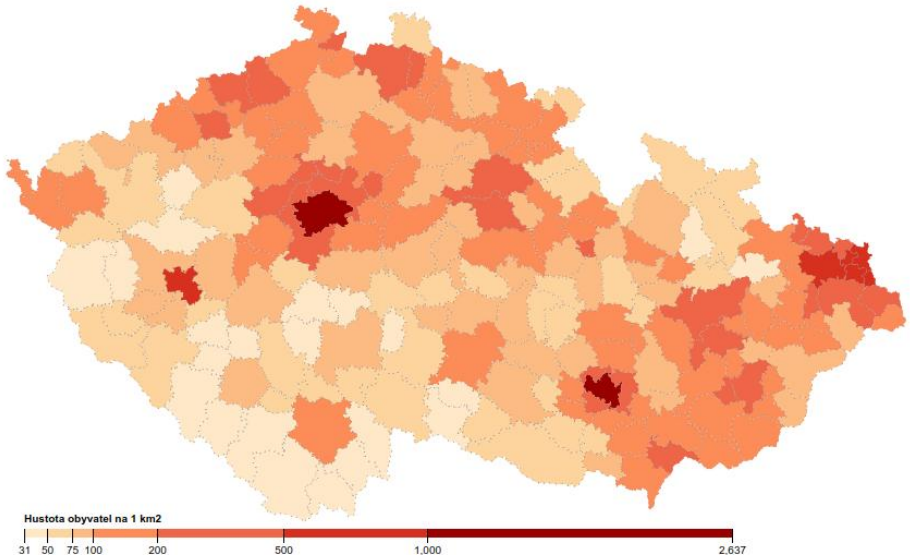


Po kliknutí na žlutý bod – krajské město zobrazí se název daného města.



Podobné mapy můžeme nalézt v učebnicích zeměpisu. Například:

Kontakt: petr.saloun@upol.cz



Výsledky a fakta:

Nejprve jsem naimportovala a postahovala potřebné knihovny.

```
import geopandas as gpd
import tkinter as tk
from tkinter import filedialog, messagebox
import folium
from folium.features import GeoJsonTooltip
from folium.plugins import Search
import json
import webbrowser
import os
```

Dále jsem vytvořila hlavní okno Tkinteru, které je ale hned skryto metodou, aby se uživateli nezobrazovalo prázdné okno. Díky tomu program může používat dialogová okna pro výběr souborů a zobrazování zpráv.

```
# skryti hlavniho okna tkinter
root = tk.Tk()
root.withdraw()
```

Následně jsem vložila dvě informační okna. První okno se zobrazí hned po spuštění programu a obsahuje informace k programu. Po odkliknutí prvního okna vyskočí další okno, které popisuje krok 1.

```
messagebox.showinfo(
    "Informace",
    "Vytvoření interaktivní mapy s počty obyvatel pro okresy i kraje.\n\n"
    "Pokračujte stisknutím OK."
)

messagebox.showinfo("Krok 1", "Vložte soubor s daty pro kraje ČR.")
```

Kontakt: petr.saloun@upol.cz

Další část kódu umožňuje uživateli vybrat dva vstupní soubory:

- GeoJSON s daty o krajích České republiky
- GeoJSON s daty o okresech České republiky

Program přes podmínku kontroluje, zda byl soubor opravdu vybrán. Pokud uživatel stiskne zrušit nebo zavřít, proměnná kraje_file zůstane prázdná. V takovém případě program vypíše hlášku do konzole a ukončí se příkazem exit(). Můžeme vkládat data typu GeoJSON a JSON. Nahraný soubor s údaji o kraji se uloží do proměnné kraje_file. Dále se zobrazí informační okno s druhý krokem a následně se provede stejný postup jako u prvního kroku – uživatel nahraje soubor, ale s údaji o okresech. Ten se pak uloží do proměnné pod názvem okresy_file.

```
23  # vyber souboru s kraji
24  kraje_file = filedialog.askopenfilename(
25      title="Vyberte GeoJSON soubor s kraji",
26      filetypes=[("GeoJSON files", "*.geojson"), ("JSON files", "*.json")]
27  )
28  if not kraje_file:
29      print("Soubor s kraji nebyl vybrán.")
30      exit()
31
32  messagebox.showinfo("Krok 2", "Vložte soubor s daty pro okresy ČR.")
33
34  # vyber souboru s okresy
35  okresy_file = filedialog.askopenfilename(
36      title="Vyberte GeoJSON soubor s okresy",
37      filetypes=[("GeoJSON files", "*.geojson"), ("JSON files", "*.json")]
38  )
39  if not okresy_file:
40      print("Soubor s okresy nebyl vybrán.")
41      exit()
42
```

Dále následuje okno pro výběr dat o krajských městech a načtení dat.

```
# cesta k souboru s mesty
messagebox.showinfo("Krok 3", "Vložte soubor s daty pro města ČR.")
mesta_file = filedialog.askopenfilename(
    title="Vyberte JSON soubor s městy",
    filetypes=[("JSON files", "*.json"), ("GeoJSON files", "*.geojson")]
)
if not mesta_file:
    print("Soubor s městy nebyl vybrán.")
    exit()

# nacteni
try:
    with open(mesta_file, "r", encoding="utf-8") as f:
        mesta_raw = json.load(f)
except Exception as e:
    messagebox.showerror("Chyba", f"Nepodařilo se načíst soubor měst:\n{mesta_file}\n\n{e}")
    raise
```

Přetvoření na seznam pro správné zobrazení souřadnic. Raději sjednotím názvy polí a převedu souřadnice na čísla.

```
# pretvoreni na seznam s name, lat, lon, admin_name
mesta = []
for m in mesta_raw:
    try:
        name = m.get("city") or m.get("name") or m.get("town")
        lat = float(m.get("lat") or m.get("latitude"))
        lon = float(m.get("lng") or m.get("lon") or m.get("longitude"))
        admin = m.get("admin_name") or m.get("region") or ""
        mesta.append({"name": name, "lat": lat, "lon": lon, "admin_name": admin})
    except Exception:
        continue

```

Funkce gpd.read file(), která načte geografická data a uloží je do objektu GeoDataFrame, který obsahuje, jak data o území, tak i jeho geometrický tvar (hranice krajů a okresů).

```
# nacteni dat
gdf_kraje = gpd.read_file(kraje_file)
gdf_okresy = gpd.read_file(okresy_file)
```

Následně je přepočítána plocha každého území z metrů čtverečních na kilometry čtvereční. Hodnota je získána vydělením sloupce shape_Area číslem 1 000 000 a uložena do nového sloupce plocha_km2.

```
# prepocet plochy na km²
gdf_kraje['plocha_km2'] = gdf_kraje['shape_Area'] / 1_000_000
gdf_okresy['plocha_km2'] = gdf_okresy['shape_Area'] / 1_000_000
```

Otevře se dialogové okno, kde si uživatel může vybrat cestu a název souboru, kam se má mapa uložit. Initialfile předvyplňuje automaticky název souboru. Ten může uživatel potvrdit nebo zadat vlastní. Dále je napsána přípona souboru a omezení typu souboru, které

uživatel může uložit – my ukládáme HTML soubor. Pokud uživatel dialogové okno zruší nebo nevybere místo uložení, program se ukončí.

```
# vyber ulozeni vysledne mapy
save_path = filedialog.asksaveasfilename(
    title="Kam uložit HTML mapu",
    initialfile="Mapa_Pocet_obyvatel.html",
    defaultextension=".html",
    filetypes=[("HTML files", "*.html")]
)
if not save_path:
    print("Cesta pro uložení nebyla vybrána.")
    exit()
```

Folium.Map() vytvoří interaktivní mapu. Střed mapy jsem nastavilla na přibližné geografické souřadnice ČR a počáteční úroveň přiblížení (zoom_start) jsem nastavila na 7. Hodnota 7 je vhodná pro zobrazení celé země s dostatečnou přehledností. Čím vyšší číslo, tím více se mapa přbližuje – detailnější zobrazení menších území.

```
# vytvoreni mapy
mapa = folium.Map(location=[49.8, 15.5], zoom_start=7)
```

Každý prvek vrstvy (jeden kraj nebo okres) je předán do funkce jako feature. Ta obsahuje atributy jako např. počet obyvatel a tvar území. Funkce style_dynamic tedy přiřazuje každému území barvu podle počtu obyvatel. Dále získám hodnotu atributu gis724060001 – počet obyvatel. Pokud atribut neexistuje, použije se výchozí hodnota 0. Try: pop = float(pop) zajišťuje, že hodnota je číslo typu float, aby šlo porovnávat.
Dále jsou vybrány barvy pro rozdělení krajů či okresů podle počtu obyvatel. Podle hodnoty je vybrána barva – menší počet → světlejší barvy, větší počet → tmavší odstíny.

Kontakt: petr.saloun@upol.cz

```python
# barvení podle populace
def style_dynamic(feature):
    pop = feature['properties'].get('gis724060001', 0)
    try:
        pop = float(pop)
    except:
        pop = 0
    if pop < 50000:
        color = "#fff7bc"
    elif pop < 100000:
        color = "#fec44f"
    elif pop < 200000:
        color = "#fe9929"
    elif pop < 500000:
        color = "#ec7014"
    elif pop < 1000000:
        color = "#cc4c02"
    else:
        color = "#8c2d04" # (největší populace)

    return {
        'fillColor': color,
        'color': 'black',
        'weight': 1,
        'fillOpacity': 0.3
    }
```

Definuji vlastnosti území po najetí myši na daný celek.

```python
def highlight_kraje(feature):
    return {
        'fillColor': "#3AA5C0D2",
        'color': 'black',
        'weight': 1,
        'fillOpacity': 0.4
    }

def highlight_okresy(feature):
    return {
        'fillColor': "#3AA5C0D2",
        'color': 'black',
        'weight': 1,
        'fillOpacity': 0.4
    }
```

Vytvořila jsem vlastní legendu pro mapu pomocí HTML a CSS. Legenda je pevně umístěná vpravo nahoře, obsahuje nadpis „Počet obyvatel kraje“ a horizontální pruh. Přechodový pruh vizuálně znázorňuje rozsah populace.

Kontakt: petr.saloun@upol.cz

```
# HTML legenda
legend_html = '''
<div style="
    position: fixed;
    top: 10px;
    right: 140px;
    width: 200px;
    padding: 0px 5px;
    z-index: 9999;
    font-size: 11px;
">
    <div style="text-align: center; margin-bottom: 2px; font-weight: bold; color: black;">
        Počet obyvatel kraje
    </div>
    <div style="
    width: 100%;
    height: 10px;
    background: linear-gradient(to right,
        #fff7bc 0%,
        #fec44f 20%,
        #fe9929 40%,
        #ec7014 60%,
        #cc4c02 80%,
        #8c2d04 100%
    );
    border: 1px solid #ccc;
"></div>
</div>
'''
```

Pomocí folium.Element() se vloží HTML kód legendy přímo do mapy. Dále nastavujeme Tooltip (malé informační okno, které se zobrazí, když uživatel) najede myší na kraj/ okres.

```
# pridani do mapy
mapa.get_root().html.add_child(folium.Element(legend_html))

# tooltipy
tooltip_kraje_fields = ['nazev', 'gis724060001', 'gis724060002', 'gis724060003', 'plocha_km2']
tooltip_kraje_aliases = ['Název kraje:', 'Celková populace:', 'Muži:', 'Ženy:', 'Plocha (km²):']

tooltip_okresy_fields = ['nazev', 'gis724060001', 'plocha_km2']
tooltip_okresy_aliases = ['Název okresu:', 'Počet obyvatel:', 'Plocha (km²):']
```

Na mapu byly přidány GeoJSON vrstvy pro kraje a okresy. Vrstva okresů je standardně skrytá a může být zobrazená pomocí ovládacího panelu vrstev. Můžeme tedy přepínat mezi zobrazením krajů a okresů. Style_function=style_dynamic znamená, že pro každý polygon (kraj) je vybrána barva podle počtu obyvatel. Highlight_function=highlight_kraje definuje, jak se polygon zvýrazní při najetí myší.

Kontakt: petr.saloun@upol.cz

```
# Vrstvy
folium.GeoJson(
    gdf_kraje,
    style_function=style_dynamic,
    highlight_function=highlight_kraje,
    tooltip=GeoJsonTooltip(fields=tooltip_kraje_fields, aliases=tooltip_kraje_aliases, localize=True),
    name="Kraje"
).add_to(mapa)

folium.GeoJson(
    gdf_okresy,
    style_function=style_dynamic,
    highlight_function=highlight_okresy,
    tooltip=GeoJsonTooltip(fields=tooltip_okresy_fields, aliases=tooltip_okresy_aliases, localize=True),
    show=False,
    name="Okresy"
).add_to(mapa)
```

Zobrazíme krajská města jako žluté tečky.

```
# mesta
mesta_geojson = {
    "type": "FeatureCollection",
    "features": [
        {
            "type": "Feature",
            "properties": {"name": m["name"]},
            "geometry": {"type": "Point", "coordinates": [m["lon"], m["lat"]]}
        }
        for m in mesta
    ]
}
mesta_layer = folium.FeatureGroup(name="Města", show=True)

for feature in mesta_geojson['features']:
    coords = feature['geometry']['coordinates']
    folium.CircleMarker(
        location=[coords[1], coords[0]],
        radius=5,
        color="#FFD700",
        fill=True,
        fill_color="#FFD700",
        fill_opacity=1,
        weight=1,
        popup=feature['properties']['name']
    ).add_to(mesta_layer)

mesta_layer.add_to(mapa)
```

Bylo přidáno ovládní vrstev. LayerControl() se zobrazí přímo v mapě a umožňuje uživateli zapínat a vypínat jednotlivé vrstvy. Funkce save() uloží celou mapu (včetně všech vrstev, tooltipů a legendy) do HTML souboru. Zobrazí se informační okno s potvrzením, že mapa

byla uložena. Zpráva uživatele informuje také o tom, že mapa se následně automaticky otevře v prohlížeči. HTML soubor se automaticky otevře ve výchozím webovém prohlížeči.

```
# pridani ovladani vrstev
folium.LayerControl().add_to(mapa)

# uloteni mapy
mapa.save(save_path)

# okno po ulozeni
messagebox.showinfo(
    "Mapa uložena",
    f"Mapa byla úspěšně uložena do:\n{save_path}\n\n"
    "Po stisknutí OK se otevře v prohlížeči."
)

# otevreni mapy v prohlizeci
webbrowser.open(f"file://{os.path.abspath(save_path)}")
```

Závěr:

Prvním krokem bylo prostudování dostupných projektů studentů z Moodlu, které mě měly inspirovat při rozhodování, která data budu chtít zpracovávat. Napadlo mě vytvořit program, který bude zobrazovat kulturní památky jako body na mapě a zároveň přidat vrstvu krajů, abychom věděli, ve kterém kraji se dané památky nachází. Procházela jsem mnoho veřejně dostupných dat, ale žádná z nich neobsahovala souřadnice, a ruční zadávání by bylo časově velmi náročné.

Při hledání jsem narazila na kompletní přehled počtu obyvatel krajů a okresů v ČR za roky 2019–2023 na stránkách ČSÚ – otevřená data. Soubor ve formátu GeoJSON obsahoval vše, co jsem potřebovala.

Cílem programu tak bylo vytvořit interaktivní mapu České republiky, která vizualizuje počet obyvatel v jednotlivých krajích a okresech.

Kód jsem tvořila ve Visual Studio Code. Musela jsem řešit problémy s knihovnami, které nefungovaly tak, jak by měly, i přesto, že byly správně nainstalované. Proto jsem program vytvářela po částech a průběžně testovala jednotlivé funkce, což se ukázalo jako efektivnější než psát celý kód najednou.

Další problém bylo nenačítání dat do mapy. Nakonec jsem zjistila, že problémy způsoboval buď pouhý překlep v názvu proměnné, nebo název proměnné, který odkazoval na úplně jiná data, než jsem potřebovala. Musela jsem projít podrobněji soubory s daty a najít požadovaný název. Tvoření programu bylo kreativní a velmi mě bavilo.

Kontakt: petr.saloun@upol.cz